Universitatea
Transilvania
din Brașov
FACULTATEA DE INGINERIE ELECTRICĂ
ȘI ȘTIINȚA CALCULATOARELOR

Documentație APM

Prof. Coordonator: S. L. KERTESZ CSABA ZOLTAN
Furtună Cristian
Iordache Denis-Ninel

BRAȘOV, 2026

CUPRINS

1 Specificații hardware **3**
1.1 Componente alese 3
1.2 Specificații finale 3
1.3 Design hardware și pcb 4
1.4 Analiză economică 4

2 Specificații software **9**
2.1 Prototip 9
2.2 Codul sursă utilizat 10

1 SPECIFICAȚII HARDWARE

1.1 COMPONENTE ALESE

INTEGRATE PARTE ANALOGICĂ

- AMC3330 – Amplificator izolat de precizie, optimizat pentru masurarea tensiunii
- ADS8920BRGET – Convertor Analog-Digital (ADC) tip SAR de inalta viteza
- THS4521IDGK – Amplificator operational complet diferential zgomot redus
- LTV-356T – Optocuploare pentru izolarea galvanica a semnalelor
- REF5045AD – Referinta de tensiune de precizie (4.5V) si deriva termica mica
- TS5A23159DGS – Comutator analogic dual SPDT (Single Pole Double Throw)
- ADuM1401xRW – Izolator digital cu 4 canale pentru liniile de date

INTEGRATE DIGITALE SI DE ALIMENTARE

- TEA1-0505 – Convertor DC/DC izolat de 1 Watt pentru alimentare și izolare galvanică
- TPS259571DSGR – Siguranta electronica (eFuse) cu monitorizare si protectie
- ATMEGA2560-16AU – Microcontroler principal pe 8 biti, familia AVR
- FT230XQ – Circuit interfata punte USB la UART serial

1.2 SPECIFICAȚII FINALE

- Backwards compatibility cu Arduino IDE (maparea pinilor este realizată în stil Arduino Mega)
- Frecvență maximă a semnalului de măsurat circa 300kHz (în funcție de configurarea ADC-ului, limită impusă de amplificatorul de intrare)
- Alimentare unică prin USB-C de 5V
- Transfer de date prin USB-C (Max. 2Mb/s)
- Intrare BNC
- Cuplaj AC/DC
- Encodere digitale pentru control al afișajului (baza de timp și scară de tensiune)

- Tensiune semnal de intrare >230Vac
- Dublă izolare galvanică pe interfața de intrare a semnalelor.

1.3 DESIGN HARDWARE ȘI PCB

Ideea acestui proiect este să creeze o izolare galvanică între partea analogică și digitală și să măsoare diferențial tensiunile aplicate la intrare astfel încât să protejeze echipamentul cu care

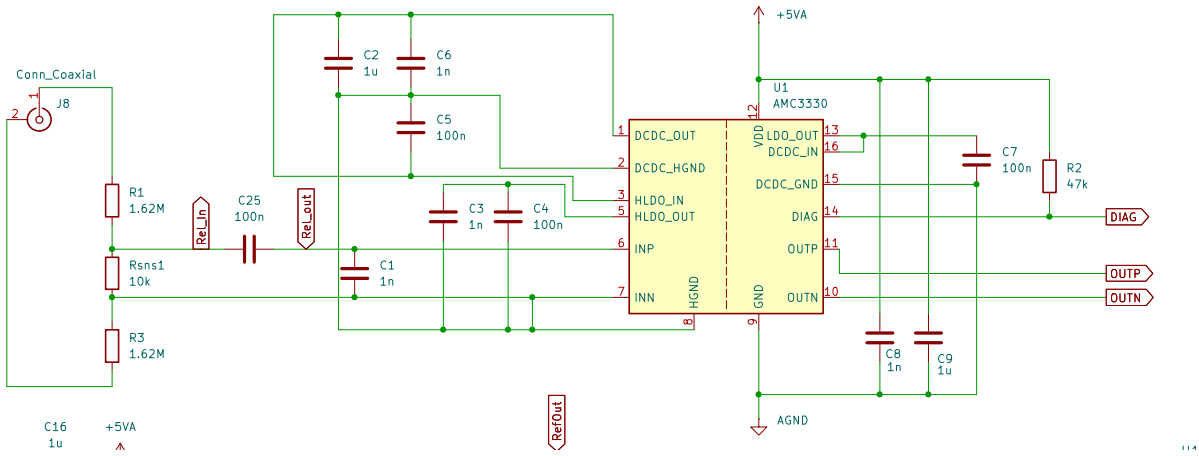

Figura 1. Schema etajului de intrare.

1.4 ANALIZĂ ECONOMICĂ

Cost PCB	Cost PCBA	Cost estimativ componente	Cost estimativ minim	Cost prototip
∼320 RON	∼1300 RON	300 + RON	600 RON	180 RON

Prin urmare, analizând cu atenție costurile din ofertele studiate, am decis să ne limităm la un PCB neasamblat + prototip realizat cu Arduino Mega 2560. Ne-am fi dorit să ne putem permite costul total de realizare a unui PCBA, dar cu măsurile de austeritate în vigoare, bursele tăiate pe perioada de vacanță și un trai tot mai scump de la o zi la alta, plus costurile de realizare a proiectului de licență, am considerat să ne oprim la o sumă totală de aprox. 500 de lei.

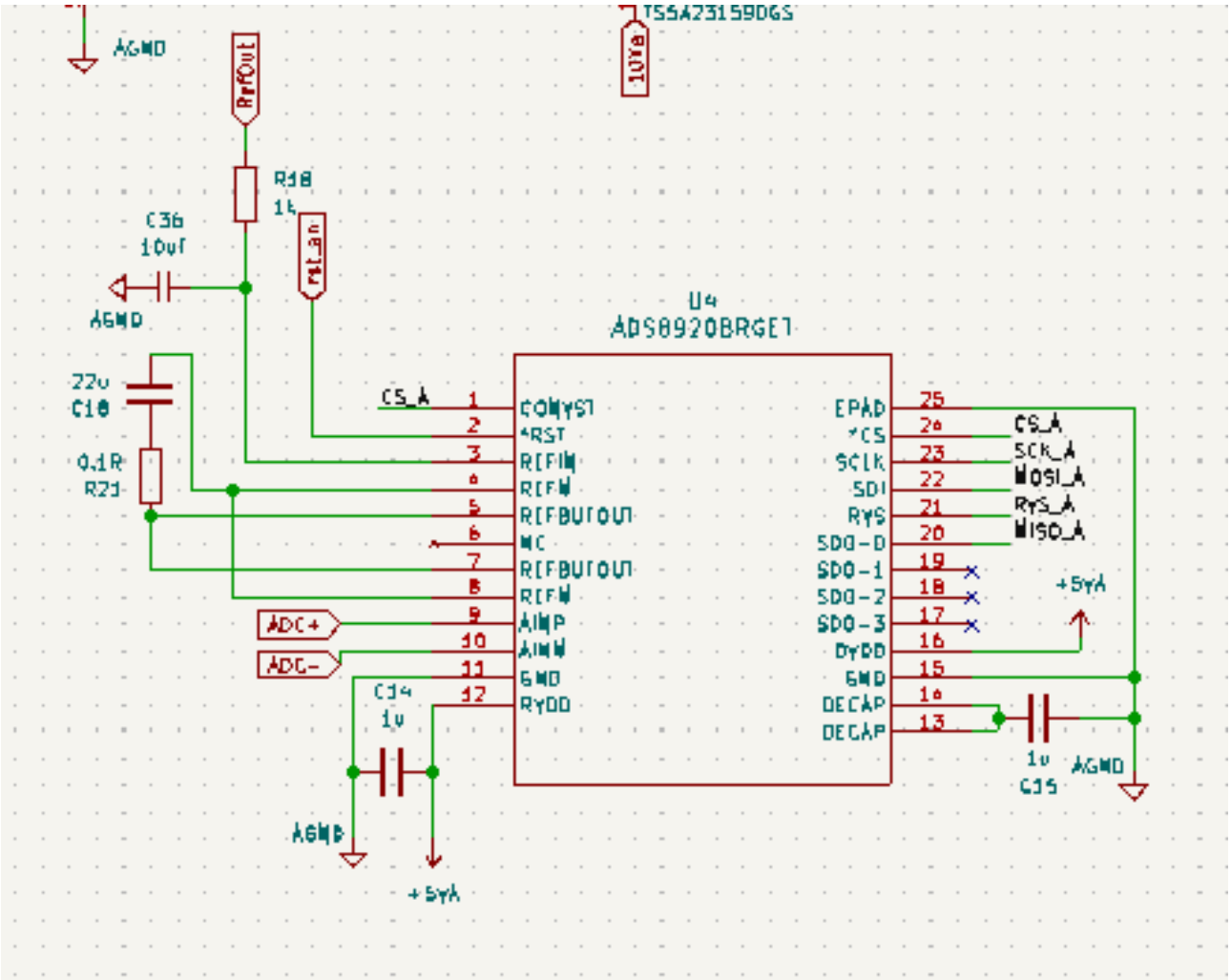

Figura 2. Schema modulului ADC.

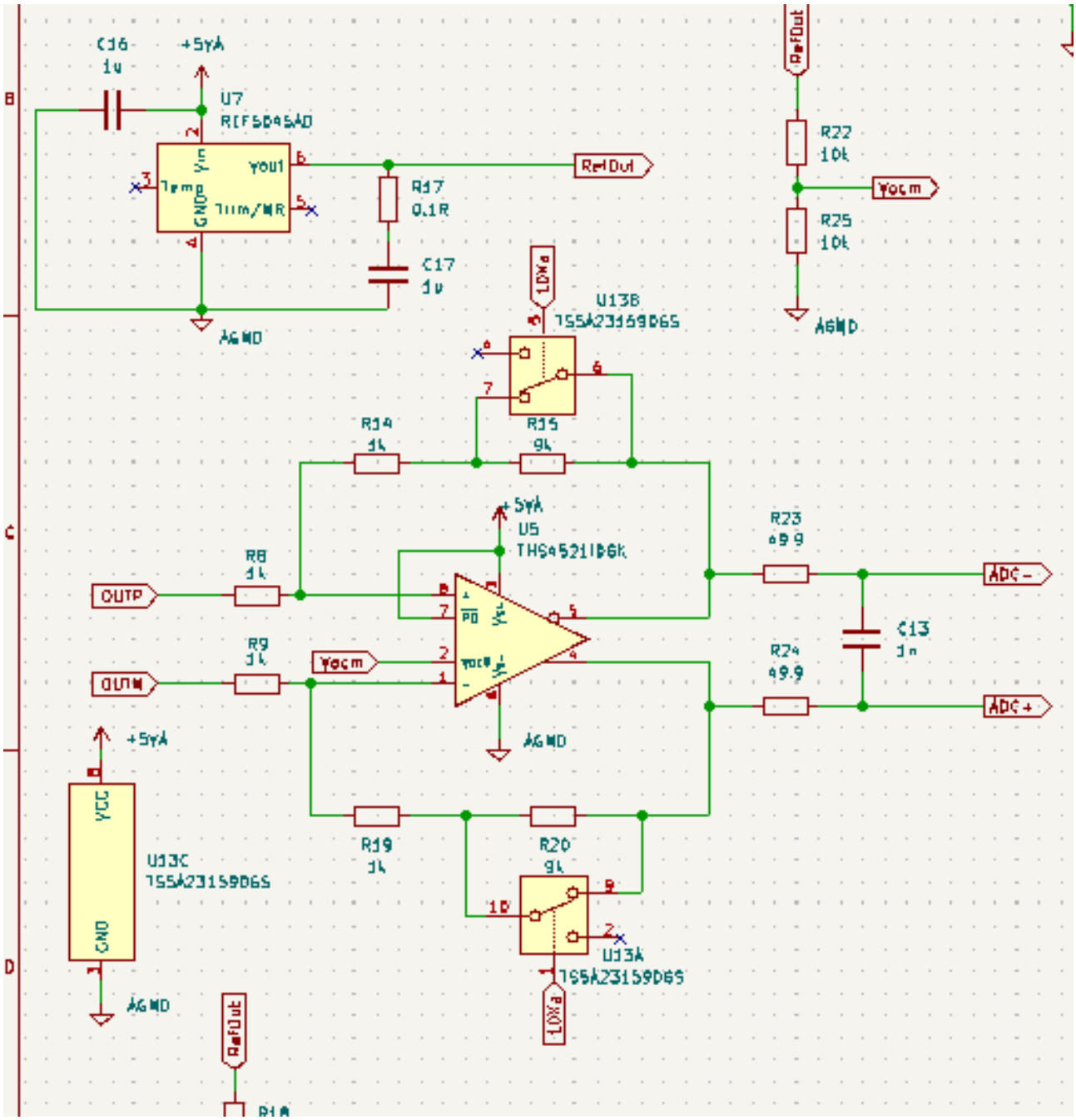

Figura 3. Schema circuitului amplificator.

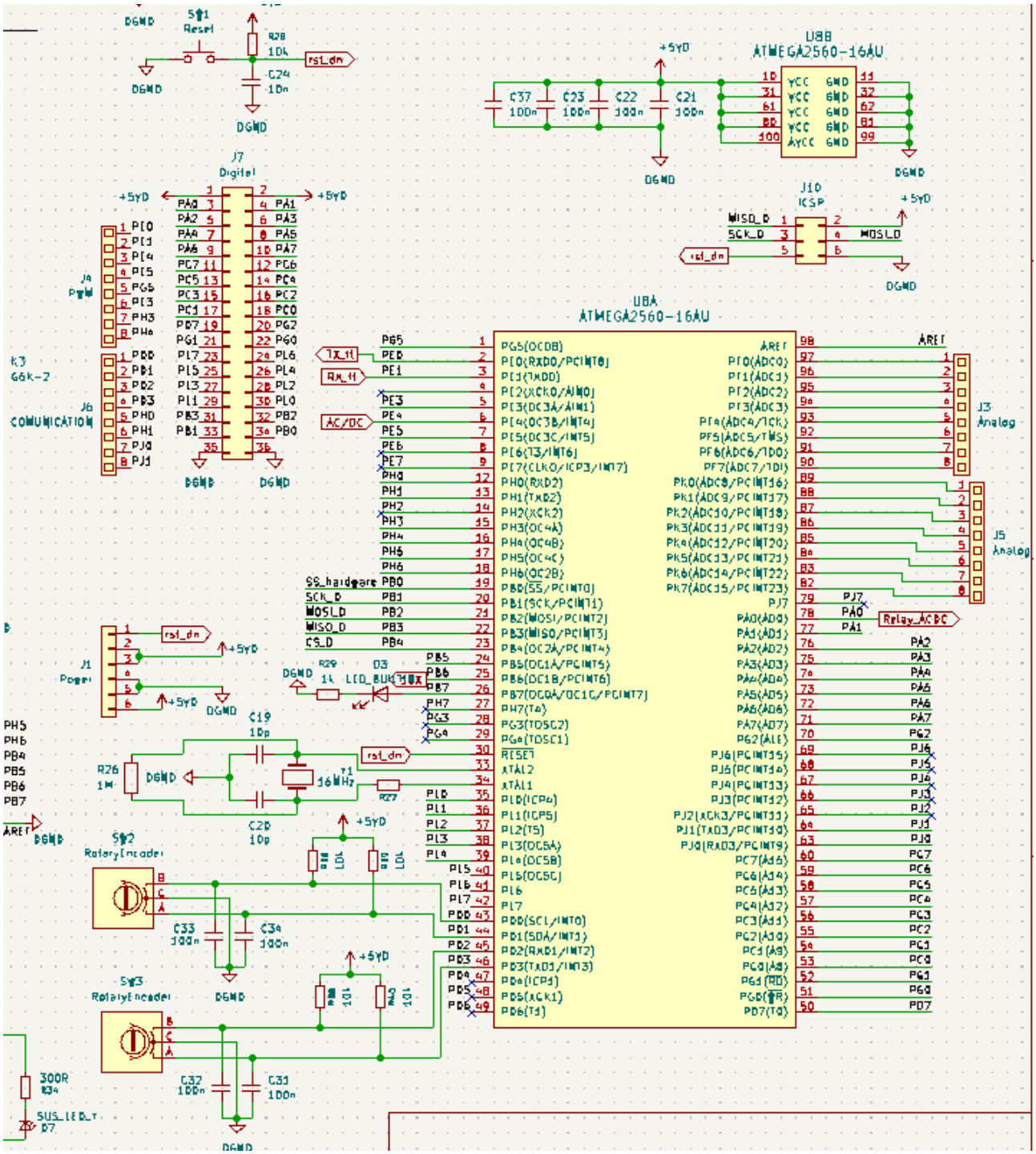

Figura 4. Configurația microcontrolerului ATmega.

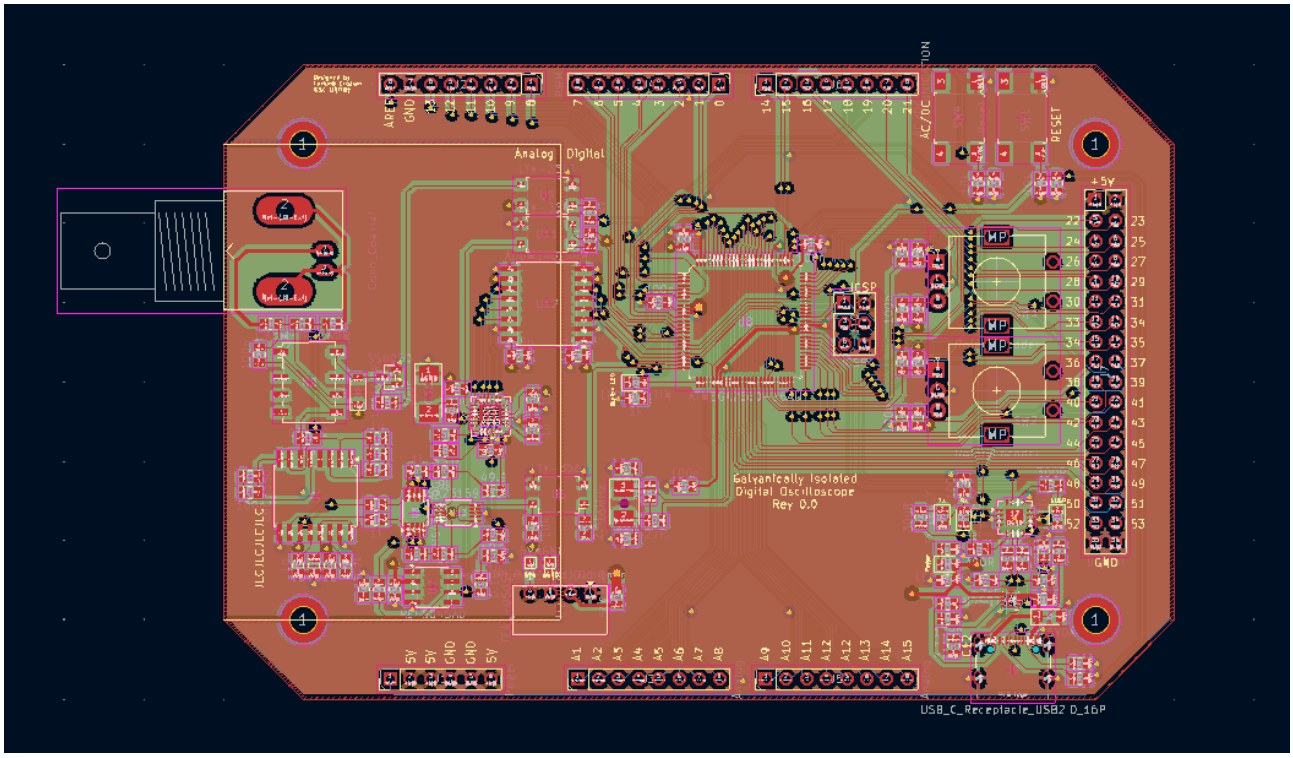

Figura 5. Design-ul (layout-ul) plăcii PCB.

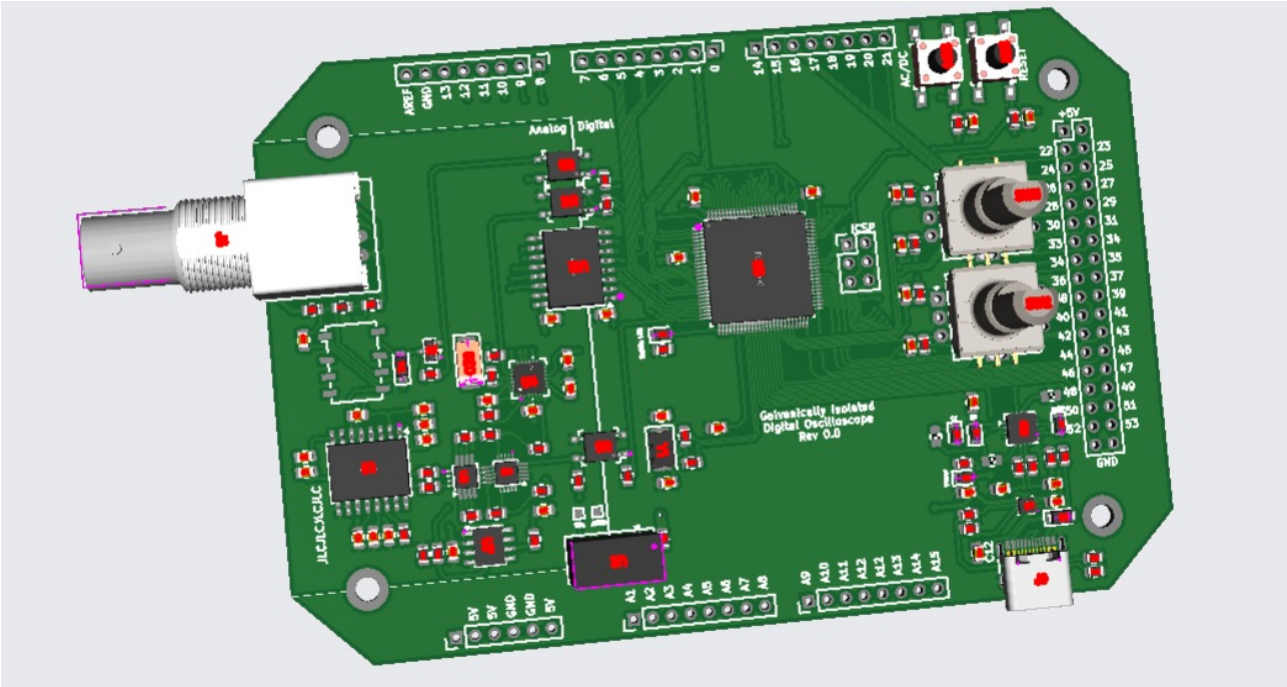

Figura 6. Fotografie cu modulul PCB asamblat.

2 SPECIFICAȚII SOFTWARE

2.1 PROTOTIP

Așa cum a fost menționat și în secțiunea de **Analiză Economică**, pentru partea de simulare ne-am limitat la un prototip realizat cu o placă de dezvoltare Arduino Mega 2560.

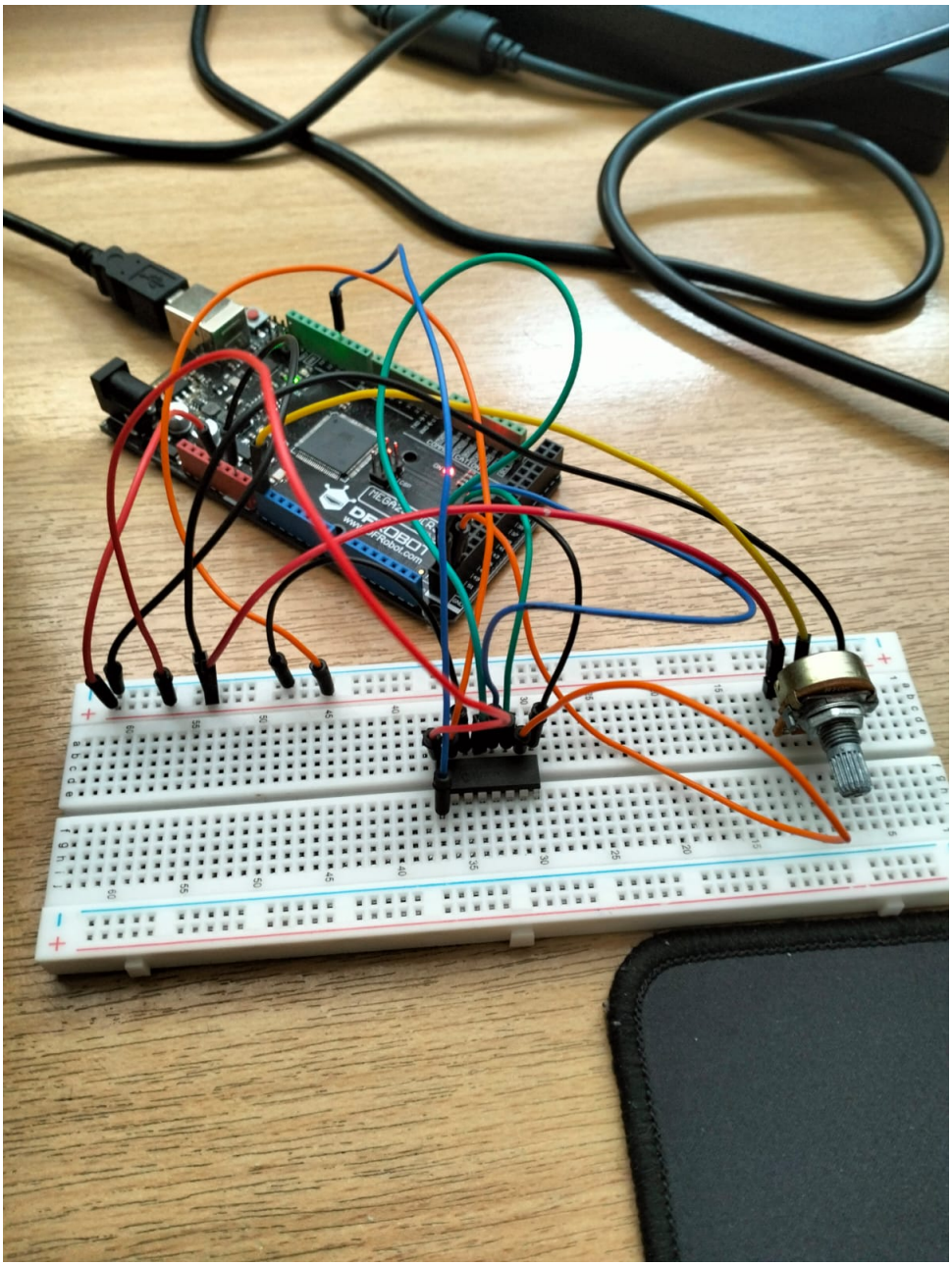

Figura 7. Fotografie cu circuitul aferent prototipului nostru implementat pe un breadboard.

Pentru circuit a fost utilizat un ADC extern pe 10 biți, MCP3008 și un potențiometru pentru setarea factorului de umplere al unui semnal dreptunghiular generat pe pinul D11 al plăcii Arduino Mega 2560.

Pinout-ul este evidențiat în următorul tabel:

Pin MCP3008	Pin ATMEGA2560	Nume pini
1	D11	CH0-D11
9	DGND	DGND - DGND
10	D53	CS-SS
11	D51	DIN-MOSI
12	D50	DOUT-MISO
13	D52	CLK-SCK
14	AGND	AGND-AGND
15	5V	VREF-5V
16	5V	VDD-5V

2.2 CODUL SURSĂ UTILIZAT

```
#ifndef SPI_H
#define SPI_H

#include <avr/io.h>

void SPI_MasterInit(void);
uint8_t SPI_MasterTransmit(uint8_t data);
void SPI_SlaveInit(void);
uint8_t SPI_SlaveReceive(void);
uint8_t SPI_Read_MCP3008_8bit(uint8_t channel);

#endif
```

```
#include <stdio.h>
#include <avr/io.h>
#include "spi.h"
#include "uart.h"

void SPI_MasterInit(void)
{
        // Set MOSI, SS and SCK output, all others input
        DDRB |= (1 << PB2) | (1 << PB1) | (1 << PB0); // MOSI - PB2, SCK - PB1
        // Enable SPI, Master, set clock rate fck/16 (was 1 << SPR0)
        PORTB |= (1 << PB0); // Keep board in Master mode
        SPCR = (1 << SPE) | (1 << MSTR) | (1 << SPR0); // SPCR - SPI control register
        SPSR |= (1 << SPI2X);
}

uint8_t SPI_MasterTransmit(uint8_t data)
{
        // Start transmission; load byte into data register
        SPDR = data; // SPDR - SPI data register
        // Wait for transmission to complete
        while (!(SPSR & (1 << SPIF)))
        ;        // SPSR - SPI status register ; SPIF - SPI Interrupt Flag
        return SPDR; // read received byte after transmission
```

```
}

void SPI_SlaveInit(void)
{
        // SET MISO output, all others input
        DDRB |= (1 << PB3); // MISO - PB3
        // Enable SPI
        SPCR = (1 << SPE); // SPCR - SPI control register
}

uint8_t SPI_SlaveReceive(void)
{
        // Wait for reception complete
        while (!(SPSR & (1 << SPIF)))
        ;        // SPSR - SPI status register ; SPIF - SPI Interrupt Flag
        return SPDR; // SPDR - SPI data register
}

// Read data from ADC
uint8_t SPI_Read_MCP3008_8bit(uint8_t channel)
{
        // CS Low
        PORTB &= ~(1 << PB0);

        // Send Start Bit (0x01)
        SPDR = 0x01;
        while (!(SPSR & (1 << SPIF)))
        ;

        // Send config bit and select channel
        SPDR = 0x80 | (channel << 4);
        while (!(SPSR & (1 << SPIF)))
        ;
        uint8_t high = SPDR;

        // Send dummy bit
        SPDR = 0x00;
        while (!(SPSR & (1 << SPIF)))
        ;
        uint8_t low = SPDR;

        // CS High
        PORTB |= (1 << PB0);

        // Concatenam high si low
        uint16_t result = ((high & 0x03) << 8) | low;
        // Shiftare la dreapta pentru a pastra doar 1 byte
        // La frecvente mari, de regula ultimii 2 biti sunt zgomotosi
        return (uint8_t)(result >> 2);
}
```

```
#define BAUD 2000000UL // desired BAUD rate

#include <avr/io.h>
#include <avr/interrupt.h>
#include <util/delay.h>
```

```
#include <stdio.h>
#include <stdint.h>
#include "uart.h"
#include "gpio.h"
#include "avr/portpins.h"
#include "pwm.h"
#include "avr/sleep.h"
#include "millis.h"
#include "adc.h"
#include "spi.h"

#define PWM_FREQ 200 // Hz
#define BATCH_SIZE 600

volatile uint8_t button_pressed = 1;

int main(void)
{

        GPIO_set_output(ONBOARD_LED); // onboard LED
        GPIO_set_output(D11);             // D11 as output

        uart_init(MYUBRR);
        SPI_MasterInit();
        millis_init();

        // Generate a square signal on pin D11
        PWM_init();
        PWM_attach(D11);
        ADC_init(ADC_REF_AVCC, ADC_PRESCALER_128);

        set_sleep_mode(SLEEP_MODE_IDLE); // Set sleep mode to power down

        EICRA |= (1 << ISC00); // Any logical change on INT0 generates interrupt
        EIMSK |= (1 << INT0);  // Enable INT0
        sei();                                  // Enable global interrupts

        while (1)
        {
                // Modify duty cycle of pwm with a potentiometer
                static uint32_t last_pwm_update = 0;

                if (millis_now() - last_pwm_update >= 50)
                {
                        uint16_t pot = ADC_read_pin(A0); // -01023
                        uint32_t duty = (uint32_t)pot * PWM_get_max(D11) / 1023;
```

```
                                        PWM_write_raw(D11, duty); // hardware PWM
                                        last_pwm_update = millis_now();
                                }

                                cli(); // clear interrupts for full focus on sending ADC data
                                // Send batch sync bytes 0xAA start byte, 0x55 stop byte, useful for python script
                                while (!(UCSR0A & (1 << UDRE0)));
                                UDR0 = 0xAA;
                                while (!(UCSR0A & (1 << UDRE0)));
                                UDR0 = 0x55;

                                // Sending BATCH_SIZE = 600 bytes; total number of bits sent = estimated 6000 bits per for
                                // Adding the start and stop bits to a byte -> 10 bits per batch
                                // 6020 if we take into consideration the sync bytes for a batch
                                for (uint16_t i = 0; i < BATCH_SIZE; i++)
                                {
                                        // Read channel 0 from ADC
                                        uint8_t val = SPI_Read_MCP3008_8bit(0);

                                        // Send byte through UART
                                        while (!(UCSR0A & (1 << UDRE0)))
                                        ;               // Wait for buffer to empty
                                        UDR0 = val; // Send the data
                                }

                                sei(); // enable interrupts
                                _delay_us(100);
                        }
                }
```

```
import sys
import time
import serial
import numpy as np
from numpy.lib.stride_tricks import sliding_window_view
from PyQt5.QtWidgets import (
QApplication, QMainWindow, QWidget, QVBoxLayout,
```

```
QHBoxLayout,
QLabel, QLineEdit, QPushButton, QDial, QGridLayout, QFrame, QGroupBox
)
from PyQt5.QtCore import QThread, pyqtSignal, Qt
import pyqtgraph as pg

# Constants
SYNC_SEQ = b"\xAA\x55" # Sync bytes
BATCH_SIZE = 600 # Size of ADC data batch
ADC_REF_VOLTAGE = 5.0  # Reference voltage of ADC = VDD (5V)

# Step values for volts/div and timescale
TIME_STEPS = [0.0001, 0.0002, 0.0005, 0.001, 0.002, 0.005, 0.010, 0.020, 0.050, 0.100]
VOLT_STEPS = [0.1, 0.2, 0.5, 1.0, 2.0, 5.0]

class SerialADCReader:
# Configure port, baud rate and timeout
def __init__(self, port="COM5", baudrate=2000000):
self.ser = serial.Serial(port, baudrate, timeout=0.01)
self.rx_buf = bytearray() # Internal buffer for received data

# Open the port
def connect(self):
if not self.ser.is_open:
self.ser.open()
self.ser.reset_input_buffer() # Reset internal buffer to get rid of junk data from previous runs

# Close the port
def disconnect(self):
if self.ser.is_open:
self.ser.close()

def read_frame(self):
# Read everything on the serial buffer
if self.ser.in_waiting > 0:
data = self.ser.read(self.ser.in_waiting)
self.rx_buf.extend(data) # Add everything from the serial buffer into our internal buffer for further processing

frames_found = []
while True:
# Search the buffer for Sync bits to find a batch
idx = self.rx_buf.find(SYNC_SEQ)
# If there are no Sync bits found and the buffer has grown in size too much
# Clear the buffer and keep only the last 100 bytes
if idx == -1:
if len(self.rx_buf) > 2 * BATCH_SIZE:
self.rx_buf = self.rx_buf[-100:]
break

```

```
56                  # Verify if there are enough bytes for a full
                       batch
57                  # If there are not, wait for full batch
58                  pkt_len = 2 + BATCH_SIZE # 2 bytes Sync + batch
                       size
59                  if len(self.rx_buf) < idx + pkt_len:
60                  break
61
62                  payload = self.rx_buf[idx + 2 : idx + pkt_len] #
                        Skip sync bytes and extract only the
                       necessary data
63                  raw_data = np.frombuffer(payload, dtype=np.uint8
                       ) # Convert into decimal values (0-255 for 8
                       bits)
64                  # Scale the values into (0 - 1020) to simulate
                       10 bit ADC
65                  # Added so I don't have to modify my program (
                       previously used to send all 10 bits of ADC
                       data)
66                  scaled_data = raw_data.astype(np.int16) * 4
67                  frames_found.extend(scaled_data)
68                  del self.rx_buf[:idx + pkt_len]
69
70                  return frames_found if frames_found else None
71
72                  # Preluare date din interfata seriala pe un
                       thread separat, fara sa blochez GUI-ul
73                  class SerialThread(QThread):
74                  samples_ready = pyqtSignal(object) # Anunta
                       programul principal ca am citit un batch
                       pentru display
75
76                  # Initializare citire buffer serial
77                  def __init__(self, port):
78                  super().__init__()
79                  self.port = port
80                  self.running = False
81                  self.reader = None
82
83                  def run(self):
84                  try:
85                  self.reader = SerialADCReader(self.port)
86                  self.reader.connect()
87                  self.running = True
88                  #In timp ce citesc datele, daca am un frame
                       complet, il trimit catre programul principal
                       pentru prelucrare
89                  while self.running:
90                  samples = self.reader.read_frame()
91                  if samples:
92                  self.samples_ready.emit(samples)
93                  else:
94                  self.msleep(1) # Mic delay daca inca nu au
                       aparut date ca sa nu blocam thread-ul inutil
95                  except Exception as e:
96                  print(f"Serial Error: {e}")  #In caz de eroare
                       pe serial read
97                  finally:
98                  if self.reader:
99                  self.reader.disconnect() # Inchidere port serial
                        (cu sau fara eroare)
100
```

```
101                 def stop(self):
102                 self.running = False
103                 self.wait()
104
105                 class MainWindow(QMainWindow):
106                 def __init__(self):
107                 super().__init__()
108
109                 self.serial_thread = None
110
111                 # Buffer circular
112                 self.capture_buffer = []
113                 self.MAX_CAPTURE = 500000 # Buffer mare
114
115                 # Variabile interne
116                 self.measured_sps = 60000.0 # Valoare default
                       pana la masura
117                 self.total_samples_received = 0
118                 self.last_time_check = time.time()
119
120                 # Scope state
121                 self.time_idx = 4   # Default 2ms (index 4 in
                       TIME_STEPS)
122                 self.volt_idx = 3   # Default 1V (index 3 in
                       VOLT_STEPS)
123                 self.trigger_lvl = 2.5
124                 self.vert_offset = 0.0
125
126                 self.last_draw_time = 0
127                 self.FPS_LIMIT = 30
128                 self.last_valid_frame = None
129                 self.trigger_lost_timer = 0
130
131                 self._setup_ui()
132                 self._setup_plot()
133                 self.update_scope_settings() # Apply initial
                       settings
134
135                 def _setup_ui(self):
136                 # Styling dial buttons
137                 dial_style = """
138                 QDial { background-color: #333; }
139                 """
140
141                 # Zona de conectare
142                 self.port_input = QLineEdit("COM5")
143                 self.port_input.setFixedWidth(80)
144                 self.btn_connect = QPushButton("ON")
145                 self.btn_connect.setCheckable(True)
146                 self.btn_connect.clicked.connect(self.
                       toggle_connection)
147                 self.btn_connect.setStyleSheet("QPushButton {
                       background-color: #444; color: white; border-
                       radius: 5px; font-weight: bold; } QPushButton:
                       checked { background-color: #00E676; color:
                       black; }")
148                 self.btn_connect.setFixedSize(50, 50)
149
150                 conn_group = QGroupBox("Input")
151                 conn_layout = QVBoxLayout()
152                 conn_layout.addWidget(QLabel("PORT"))
153                 conn_layout.addWidget(self.port_input)
```

```
154                     conn_layout.addWidget(self.btn_connect)
155                     conn_layout.addStretch()
156                     conn_group.setLayout(conn_layout)
157
158                     # Zona Verticala (Volts/Div & Position - offset)
159                     self.dial_volts = QDial()
160                     self.dial_volts.setRange(0, len(VOLT_STEPS)-1)
161                     self.dial_volts.setValue(self.volt_idx)
162                     self.dial_volts.setNotchesVisible(True)
163                     self.dial_volts.valueChanged.connect(self.
                           update_scope_settings)
164                     self.lbl_volts = QLabel("1 V")
165                     self.lbl_volts.setAlignment(Qt.AlignCenter)
166
167                     self.dial_offset = QDial()
168                     self.dial_offset.setRange(-60, 60) # +/- 6.0
                           Diviziuni
169                     self.dial_offset.setValue(0)
170                     self.dial_offset.setNotchesVisible(True)
171                     self.dial_offset.valueChanged.connect(self.
                           update_scope_settings)
172                     self.lbl_offset = QLabel("Pos: 0 Div")
173                     self.lbl_offset.setAlignment(Qt.AlignCenter)
174
175                     vert_group = QGroupBox("Vertical")
176                     vert_layout = QVBoxLayout()
177                     vert_layout.addWidget(QLabel("SCALE"))
178                     vert_layout.addWidget(self.dial_volts)
179                     vert_layout.addWidget(self.lbl_volts)
180                     vert_layout.addWidget(QLabel("POSITION"))
181                     vert_layout.addWidget(self.dial_offset)
182                     vert_layout.addWidget(self.lbl_offset)
183                     vert_group.setLayout(vert_layout)
184
185                     # Zona ăOrizontal (Time/Div)
186                     self.dial_time = QDial()
187                     self.dial_time.setRange(0, len(TIME_STEPS)-1)
188                     self.dial_time.setValue(self.time_idx)
189                     self.dial_time.setNotchesVisible(True)
190                     self.dial_time.valueChanged.connect(self.
                           update_scope_settings)
191                     self.lbl_time = QLabel("2 ms")
192                     self.lbl_time.setAlignment(Qt.AlignCenter)
193
194                     horiz_group = QGroupBox("Horizontal")
195                     horiz_layout = QVBoxLayout()
196                     horiz_layout.addWidget(QLabel("TIME/DIV"))
197                     horiz_layout.addWidget(self.dial_time)
198                     horiz_layout.addWidget(self.lbl_time)
199                     horiz_layout.addStretch()
200                     horiz_group.setLayout(horiz_layout)
201
202                     # Zona Trigger
203                     self.dial_trig = QDial()
204                     self.dial_trig.setRange(0, 50) # 0.0 la 5.0V
205                     self.dial_trig.setValue(25)
206                     self.dial_trig.setNotchesVisible(True)
207                     self.dial_trig.valueChanged.connect(self.
                           update_scope_settings)
208                     self.lbl_trig_val = QLabel("T: 2.5V")
209                     self.lbl_trig_val.setAlignment(Qt.AlignCenter)
```

```
210
211                 self.lbl_trig_status = QLabel("NO SIG")
212                 self.lbl_trig_status.setStyleSheet("color: gray;
                        font-weight: bold; border: 1px solid gray;
                       padding: 2px;")
213                 self.lbl_trig_status.setAlignment(Qt.AlignCenter
                       )
214
215                 trig_group = QGroupBox("Trigger")
216                 trig_layout = QVBoxLayout()
217                 trig_layout.addWidget(QLabel("LEVEL"))
218                 trig_layout.addWidget(self.dial_trig)
219                 trig_layout.addWidget(self.lbl_trig_val)
220                 trig_layout.addWidget(self.lbl_trig_status)
221                 trig_group.setLayout(trig_layout)
222
223                 # Panou Control (stanga)
224                 ctrl_layout = QVBoxLayout()
225                 ctrl_layout.addWidget(conn_group)
226                 ctrl_layout.addWidget(vert_group)
227                 ctrl_layout.addWidget(horiz_group)
228                 ctrl_layout.addWidget(trig_group)
229                 ctrl_layout.addStretch()
230
231                 ctrl_widget = QWidget()
232                 ctrl_widget.setLayout(ctrl_layout)
233                 ctrl_widget.setFixedWidth(180) # Latime fixa a
                       panoului
234
235                 # Plot Area
236                 self.plot_widget = pg.GraphicsLayoutWidget()
237                 self.plot_widget.setBackground('#111') #
                       Background inchis
238                 self.plot = self.plot_widget.addPlot(title="
                       NIBIRUSCOPE")
239                 self.curve = self.plot.plot(pen=pg.mkPen('#00
                       E676', width=2))
240
241                 # Setup Grid
242                 self.plot.showGrid(x=True, y=True, alpha=0.3)
243                 self.plot.setLabel('bottom', 'Time', units='s')
244                 self.plot.setLabel('left', 'Voltage', units='V')
245
246                 # Main Layout
247                 main_layout = QHBoxLayout()
248                 main_layout.addWidget(ctrl_widget)
249                 main_layout.addWidget(self.plot_widget)
250
251                 container = QWidget()
252                 container.setLayout(main_layout)
253                 self.setCentralWidget(container)
254                 self.setWindowTitle("NibiruScope")
255                 self.resize(1200, 700)
256
257                 def _setup_plot(self):
258                 pg.setConfigOptions(antialias=False)
259
260                 def update_scope_settings(self):
261                 # Timebase
262                 idx_t = self.dial_time.value()
263                 time_val = TIME_STEPS[idx_t]
264
```

```
265                     if time_val < 0.001:
266                     self.lbl_time.setText(f"{time_val*1000000:.0f}
                           µs")
267                     elif time_val < 1.0:
268                     self.lbl_time.setText(f"{time_val*1000:.0f} ms")
269                     else:
270                     self.lbl_time.setText(f"{time_val:.2f} s")
271
272                     # Volts/div
273                     idx_v = self.dial_volts.value()
274                     volt_per_div = VOLT_STEPS[idx_v]
275                     self.lbl_volts.setText(f"{volt_per_div} V")
276
277                     # Position - Offset
278                     # Modifica valoarea dial-ului in diviziuni
279                     offset_in_divs = self.dial_offset.value() / 10.0
280
281                     # Scalare pentru diferite valori de volts/div
282                     self.vert_offset = offset_in_divs * volt_per_div
283
284                     # Afisare label offset scalat in volti pentru
                           dial
285                     self.lbl_offset.setText(f"Pos: {self.vert_offset
                           :+.1f}V")
286
287                     # Trigger
288                     trig_raw = self.dial_trig.value()
289                     self.trigger_lvl = trig_raw / 10.0
290                     self.lbl_trig_val.setText(f"T: {self.trigger_lvl
                           :.1f}V")
291
292
293                     # Axa X
294                     self.plot.setXRange(0, time_val * 10, padding=0)
295
296                     # Axa Y:
297                     half_screen_volts = volt_per_div * 4
298                     self.plot.setYRange(-half_screen_volts,
                           half_screen_volts, padding=0)
299
300                     # Functionalitate on/off
301                     def toggle_connection(self):
302                     if self.btn_connect.isChecked():
303                     port = self.port_input.text()
304                     self.serial_thread = SerialThread(port)
305                     self.serial_thread.samples_ready.connect(self.
                           process_data)
306                     self.serial_thread.start()
307                     self.btn_connect.setText("OFF")
308                     else:
309                     if self.serial_thread:
310                     self.serial_thread.stop()
311                     self.serial_thread = None
312                     self.btn_connect.setText("ON")
313
314                     def process_data(self, samples):
315                     self.total_samples_received += len(samples)
316                     now = time.time()
317                     if now - self.last_time_check >= 1.0:
318                     self.measured_sps = self.total_samples_received
                           / (now - self.last_time_check)
```

```
319                 self.total_samples_received = 0
320                 self.last_time_check = now
321                 # ăActualizm titlul ferestrei
322                 # self.plot.setTitle(f"NIBIRU SCOPE - Sampling:
                       {self.measured_sps/1000:.1f} kSPS")
323
324                 # Buffer Management
325                 self.capture_buffer.extend(samples)
326                 if len(self.capture_buffer) > self.MAX_CAPTURE:
327                 self.capture_buffer = self.capture_buffer[-self.
                       MAX_CAPTURE:]
328
329                 # Limitare FPS
330                 if now - self.last_draw_time < (1.0 / self.
                       FPS_LIMIT):
331                 return
332
333
334                 time_per_div = TIME_STEPS[self.dial_time.value()
                       ]
335                 total_time_screen = time_per_div * 10
336
337                 n_display_points = int(total_time_screen * self.
                       measured_sps)
338
339                 n_display_points = max(10, min(n_display_points,
                        len(self.capture_buffer)))
340
341                 if len(self.capture_buffer) < n_display_points *
                        2:
342                 return
343
344                 # Procesare date
345                 # ăLum un chunk mare din ăcoad pentru a ăcuta
                       trigger-ul
346                 # ăLum dublu cât ne trebuie ca ăs avem loc de
                       ămanevr
347                 search_window = int(n_display_points * 2.5)
348                 if search_window > len(self.capture_buffer):
349                 search_window = len(self.capture_buffer)
350
351                 raw_data = np.array(self.capture_buffer[-
                       search_window:], dtype=np.int16)
352
353                 # Filtru Median (doar ădac avem destule puncte)
354                 if len(raw_data) > 5:
355                 window = sliding_window_view(raw_data,
                       window_shape=3)
356                 filtered_data = np.median(window, axis=1)
357                 else:
358                 filtered_data = raw_data
359
360                 # --- 6. TRIGGER LOGIC ---
361                 trig_adc = int((self.trigger_lvl / 5.0) * 1023)
362
363                 # Cautare rising edge
364                 binary = (filtered_data >= trig_adc).astype(np.
                       int8)
365                 edges = np.diff(binary)
366                 trigger_indices = np.where(edges == 1)[0]
367
```

```
                    frame_to_draw = None
                    found_trigger = False

                    if len(trigger_indices) > 0:

                    for idx in reversed(trigger_indices):
                    if idx + n_display_points < len(filtered_data):
                    if np.all(filtered_data[idx+1 : idx+4] > (
                       trig_adc - 20)):
                    frame_to_draw = filtered_data[idx : idx +
                       n_display_points]
                    found_trigger = True

                    self.lbl_trig_status.setText("LOCKED")
                    self.lbl_trig_status.setStyleSheet("color: #00
                       E676; border: 1px solid #00E676; font-weight:
                       bold;")
                    self.trigger_lost_timer = now
                    break

                    # Auto roll
                    if not found_trigger:
                    if now - self.trigger_lost_timer > 0.3:
                    self.lbl_trig_status.setText("AUTO")
                    self.lbl_trig_status.setStyleSheet("color:
                       orange; border: 1px solid orange;")
                    # Luam ultimele date disponibile
                    if len(filtered_data) >= n_display_points:
                    frame_to_draw = filtered_data[-n_display_points
                       :]

                    # Desenare
                    if frame_to_draw is not None:
                    self.last_draw_time = now

                    # Conversie in Volti
                    real_volts = (frame_to_draw / 1023.0) * 5.0

                    display_volts = (real_volts - 2.5) + self.
                       vert_offset

                    # Generare axa timp
                    total_time_screen = time_per_div * 10
                    t = np.linspace(0, total_time_screen, len(
                       display_volts))

                    self.curve.setData(t, display_volts)

                    if __name__ == "__main__":
                    app = QApplication(sys.argv)
                    window = MainWindow()
                    window.show()
                    sys.exit(app.exec_())
```